

BIM INGENIEROS • BIMROCKET + IOT

Lección 1

Del sensor REST al objeto BIM conectado

Cómo llevar una medición IoT simulada hasta una sala BIM que muestra, colorea y valida el dato.

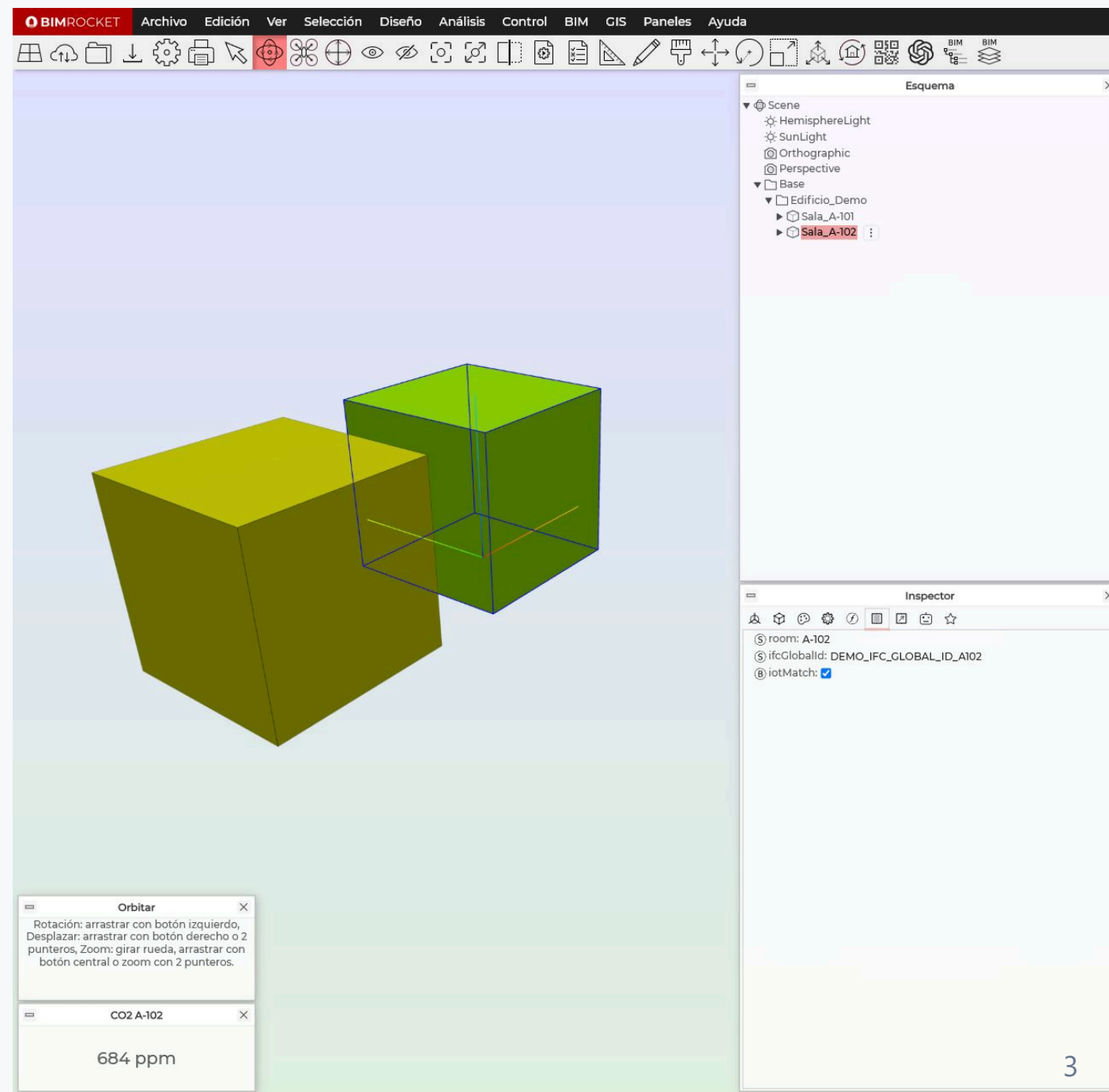
Objetivo de la lección

Entender el primer flujo completo:

```
sensor/API → dato JSON → objeto BIM → visualización → validación
```

Qué vas a construir

- Una API REST simulada con varias salas.
- Dos objetos BIM: `Sala_A-101` y `Sala_A-102`.
- Un panel de CO₂ por sala.
- Color automático según CO₂, sensor offline o identidad incorrecta.



El sensor devuelve JSON

Respuesta REST — Sala A-102

```
{  
  "room": "A-102",  
  "ifcGlobalId": "DEMO_IFC_GLOBAL_ID_A102",  
  "temperature": 22.5,  
  "co2": 815,  
  "status": "online"  
}
```

La API devuelve identidad, medición y estado.

Camino del dato

Camino del dato IoT en BIMROCKET



La geometría se convierte en un objeto con identidad, datos vivos y reglas de confianza.

Identidad BIM + IoT

La clave no es solo leer datos.

La clave es saber **a qué objeto BIM pertenecen.**

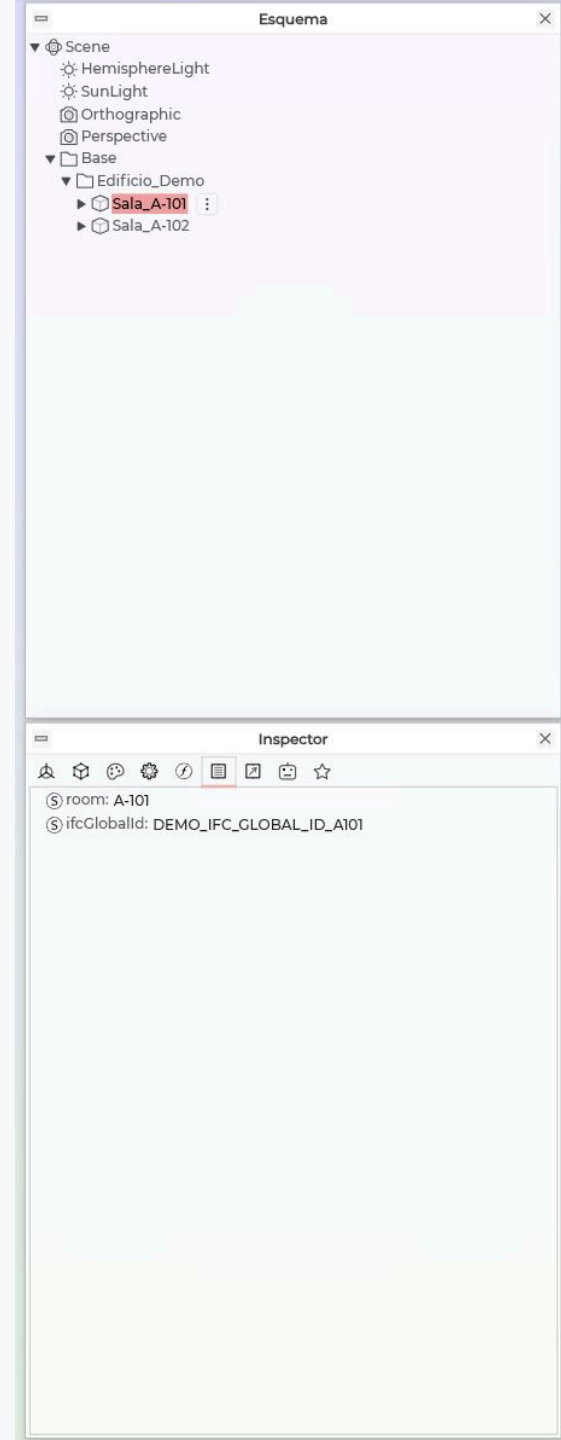
A-101: el objeto sabe quién es

En el esquema está seleccionada `Sala_A-101`.

En el inspector, pestaña de propiedades, vemos:

```
room = A-101  
ifcGlobalId = DEMO_IFC_GLOBAL_ID_A101
```

Ese `room` será el identificador usado para pedir datos a la API.



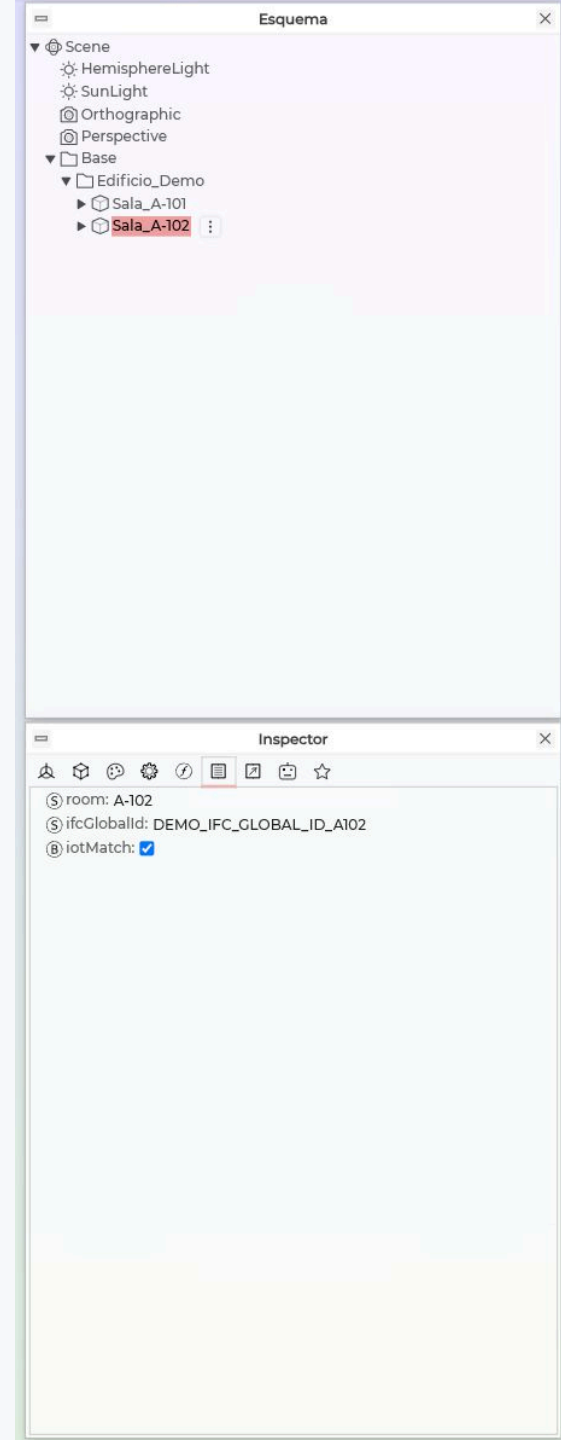
A-102: misma lógica, otro objeto

Ahora está seleccionada `Sala_A-102`.

La estructura es la misma, pero los valores cambian:

```
room = A-102  
ifcGlobalId = DEMO_IFC_GLOBAL_ID_A102
```

Esto permite reutilizar reglas sin duplicar lógica a mano.



Leer datos desde la API

BIMROCKET pregunta cada cierto tiempo a una URL y recibe la respuesta JSON.

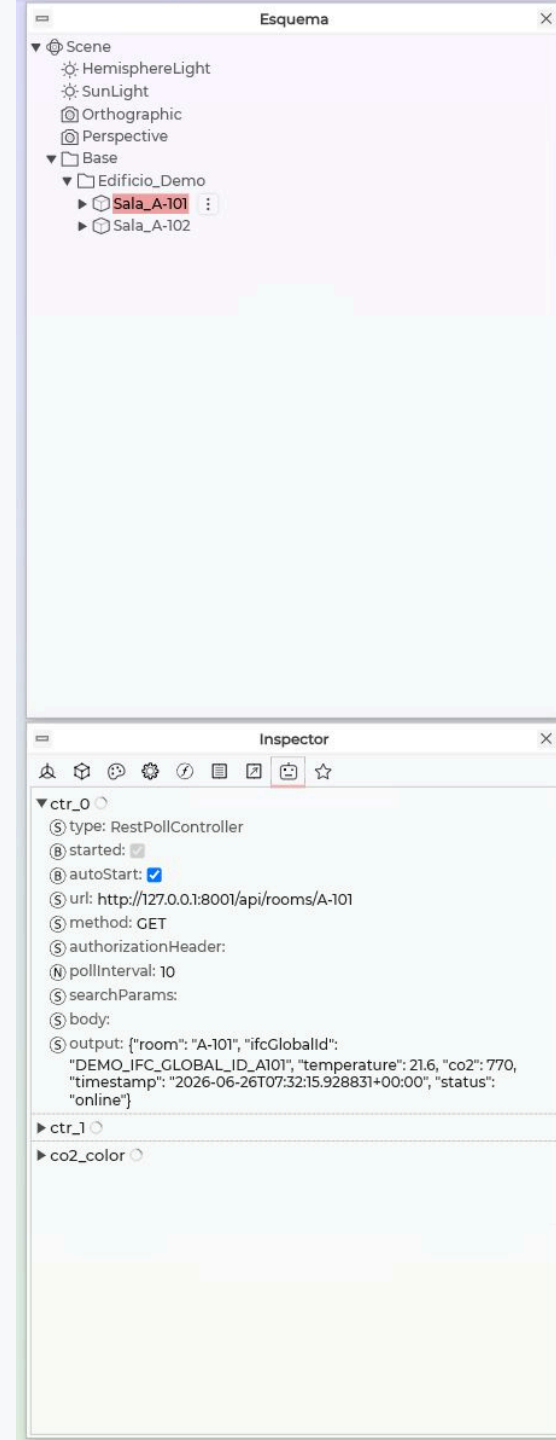
RestPollController en A-101

En `Sala_A-101`, el controlador REST consulta:

```
http://127.0.0.1:8001/api/rooms/A-101
```

El campo `output` muestra la respuesta recibida:

```
room, ifcGlobalId, temperature, co2, timestamp, status
```



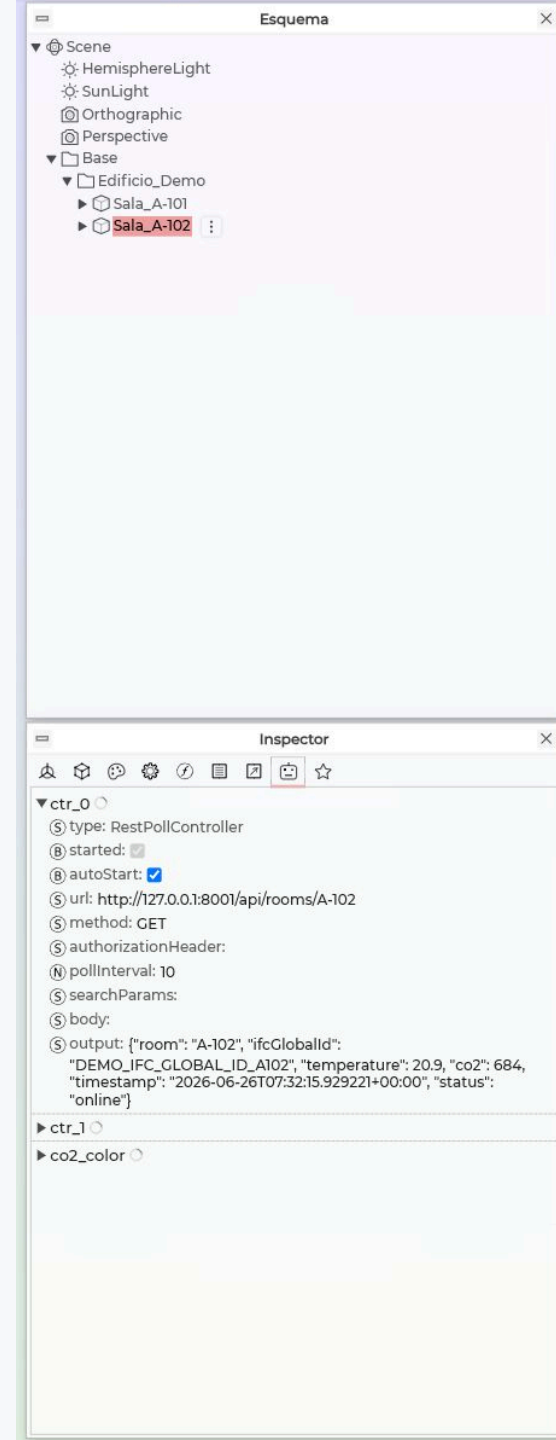
RestPollController en A-102

En `Sala_A-102`, el mismo controlador consulta:

```
http://127.0.0.1:8001/api/rooms/A-102
```

La idea importante:

cada objeto BIM pregunta por su propio dato.



URL dinámica del sensor

URL dinámica del sensor

`"http://127.0.0.1:8001/api/rooms/" + object.userData.room`

Texto fijo + identidad de la sala actual

`Sala_A-102 → /api/rooms/A-102`

`Sala_A-101 → /api/rooms/A-101`

Fórmula en BIMROCKET

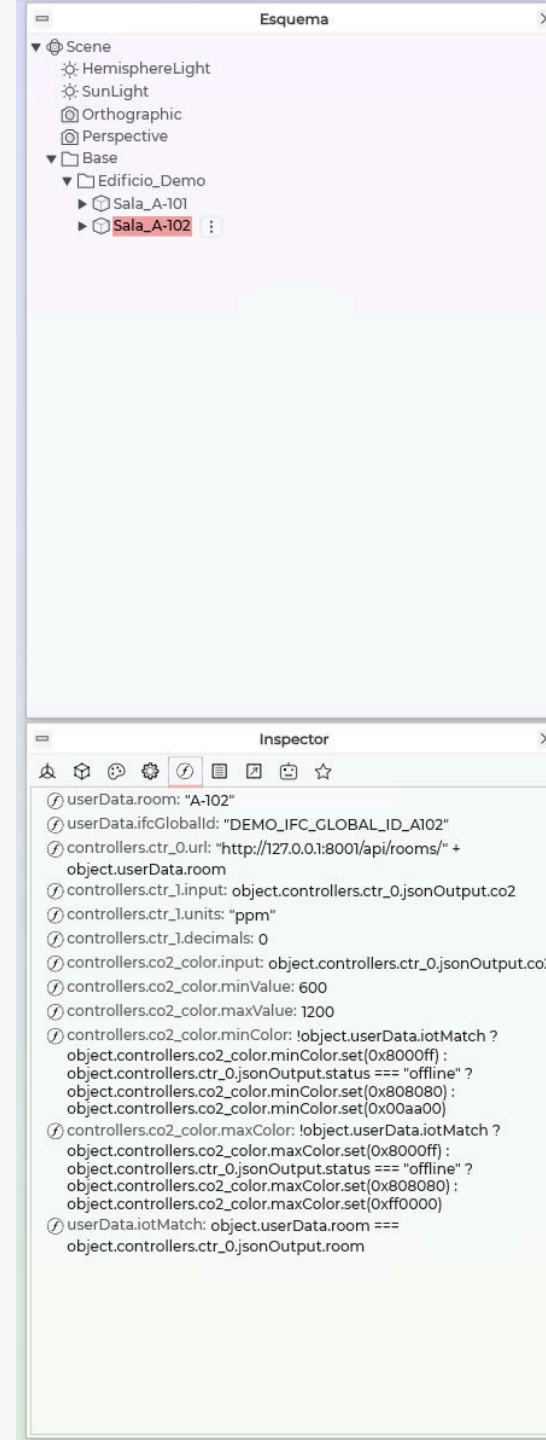
Aquí está seleccionada **Sala_A-102** y está abierta la pestaña de Fórmulas.

La URL no está escrita como **A-102** fijo.

Usa:

```
object.userData.room
```

La misma fórmula sirve para **A-101** y **A-102**.



Mostrar el CO₂

Leer el dato no basta.

Tenemos que convertirlo en información visible para el usuario.

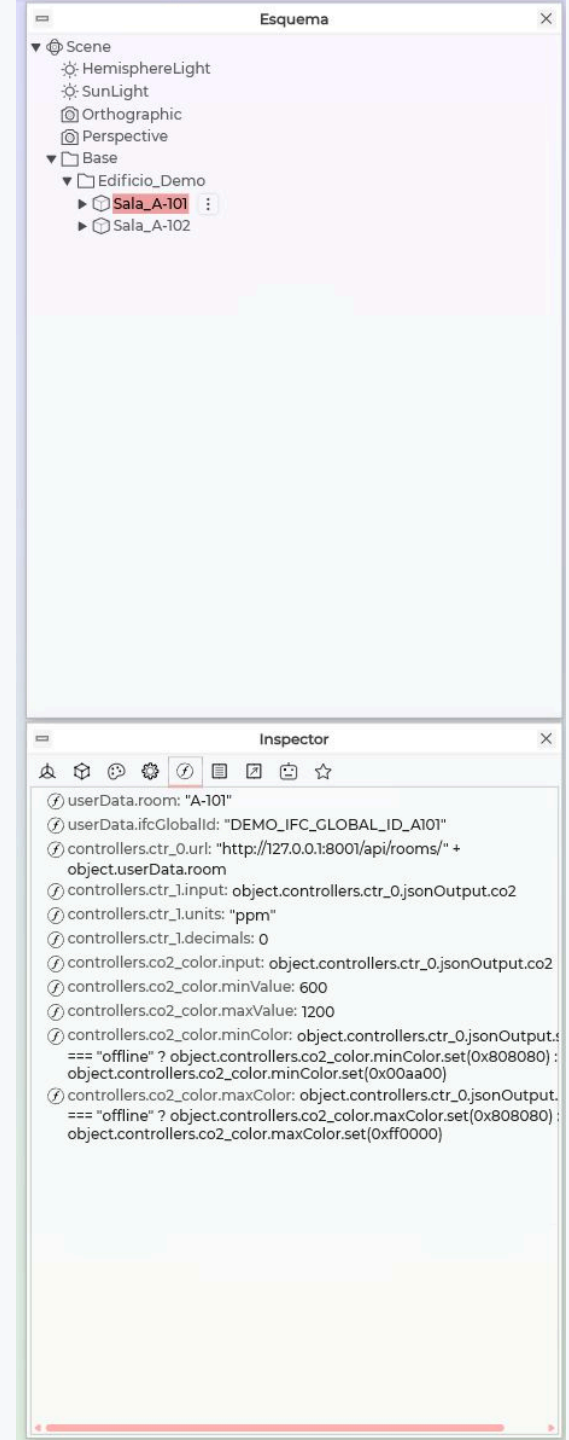
Fórmula para mostrar CO₂

En `Sala_A-101`, el `DisplayController` toma el CO₂ del JSON:

```
object.controllers.ctr_0.jsonOutput.co2
```

Traducción:

del resultado recibido por REST, usa el campo `co2`.

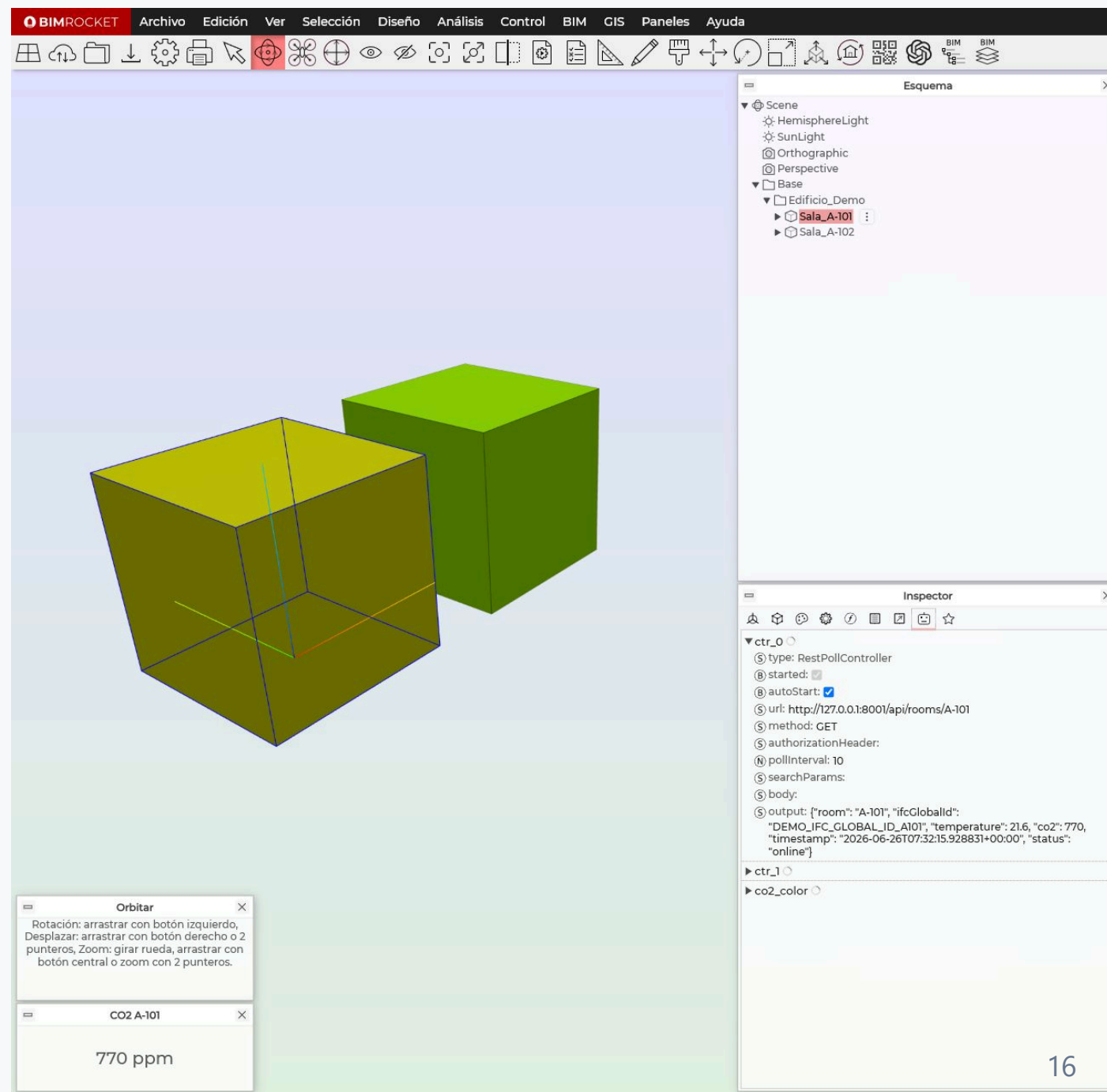


Resultado visible

El panel inferior muestra:

C02 A-101
770 ppm

La sala ya no es solo geometría:
ahora enseña un dato vivo.



Validar confianza del dato

Un dato puede llegar, pero no pertenecer al objeto correcto.

iotMatch

Validación de identidad IoT

```
object.userData.room === object.controllers.ctr_0.jsonOutput.room
```

Pregunta:

¿La sala BIM coincide con la sala que dice la API?

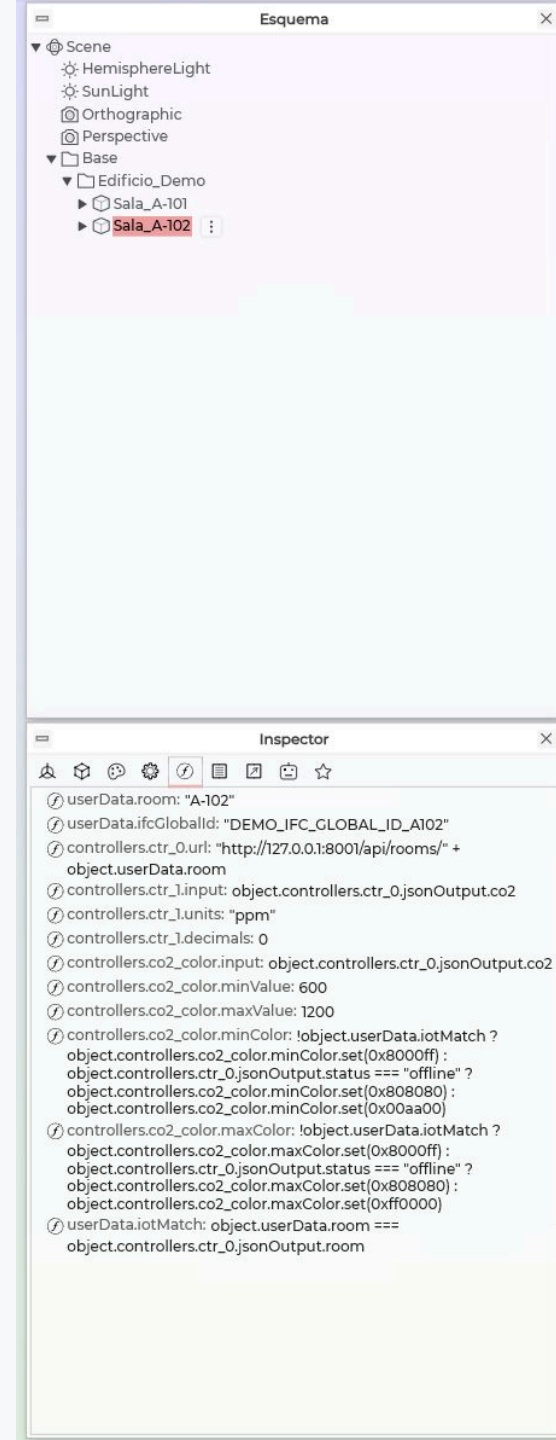
true = dato confiable · false = alerta visual morada

iotMatch en A-102

En la pestaña de Fórmulas se ve la comparación:

```
object.userData.room === object.controllers.ctr_0.jsonOutput.room
```

Si ambas partes coinciden, el dato es confiable para esa sala.



Estados visuales

Estados visuales del dato

Todo correcto

`status = online`
`iotMatch = true`
color según CO₂

Sensor offline

`status = offline`
`iotMatch = true`
gris

Identidad incorrecta

`iotMatch = false`
morado
prioridad máxima

Prioridad: morado → gris → CO₂

Reglas finales de color

Para `minColor`:

```
!object.userData.iotMatch
  ? object.controllers.co2_color.minColor.set(0x8000ff)
  : object.controllers.ctr_0.jsonOutput.status === "offline"
    ? object.controllers.co2_color.minColor.set(0x808080)
    : object.controllers.co2_color.minColor.set(0x00aa00)
```

Prioridad de confianza

Prioridad	Condición	Color	Significado
1	<code>iotMatch = false</code>	Morado	Dato no confiable
2	<code>status = offline</code>	Gris	Sensor no disponible
3	Todo correcto	Verde-rojo	CO ₂ válido

Pruebas realizadas

No basta con configurar.
Hay que probar cada estado.

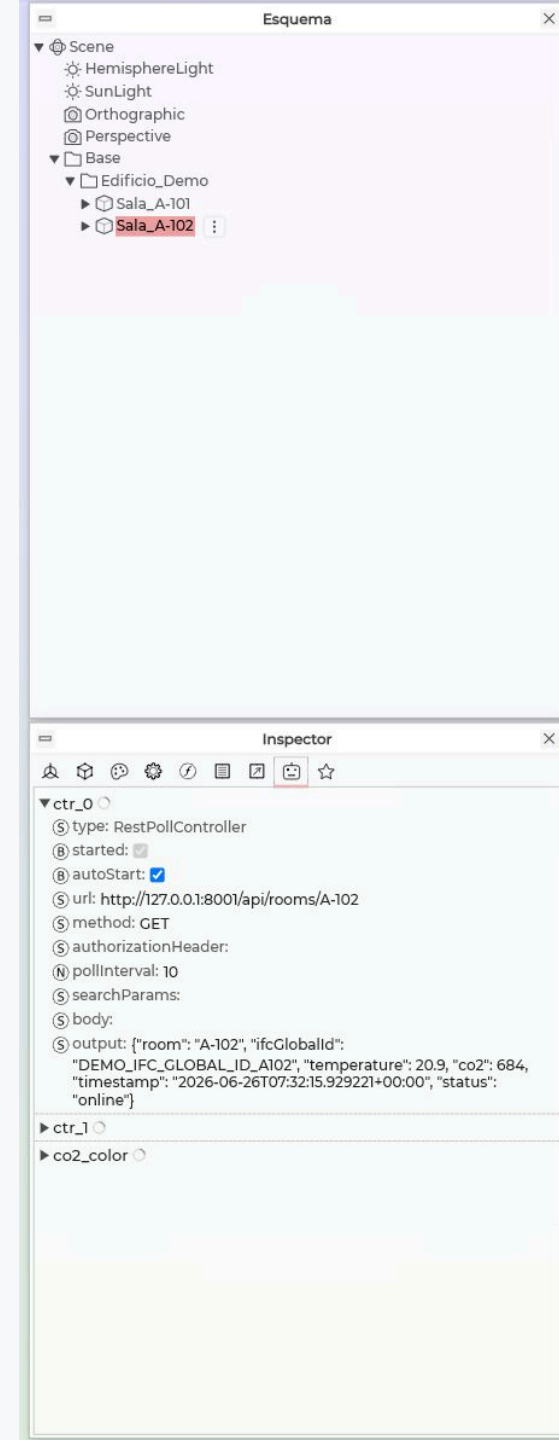
Prueba 1 — Caso normal

En el controlador REST vemos:

```
status = online  
room = A-102
```

Resultado:

```
iotMatch = true  
color = según CO2
```



Prueba 2 — Sensor offline

URL temporal usada:

```
"http://127.0.0.1:8001/api/rooms/" + object.userData.room + "?offline=1"
```

Resultado esperado:

```
status = offline  
iotMatch = true  
color = gris
```

Esta prueba comprueba disponibilidad del sensor.

Prueba 3 — Identidad incorrecta

Fórmula temporal usada:

```
"A-999" === object.controllers.ctr_0.jsonOutput.room
```

Resultado esperado:

```
iotMatch = false  
color = morado
```

Esta prueba comprueba confianza del dato.

Modelo final

Archivo de referencia:

```
examples/bimrocket-models/lab-03-dos-salas-iot.brf
```

Contiene:

- dos salas conectadas;
- URL dinámica;
- paneles de CO₂;
- validación `iotMatch` ;
- estados visuales de confianza.

Cierre conceptual

Un gemelo digital no es solo un modelo 3D.
Es geometría + identidad + datos vivos + reglas de confianza.

BIM INGENIEROS

Siguiente paso

Del sensor simulado al sensor real con ESP32.

La lógica aprendida se mantiene. Solo cambia la fuente del dato.